

8/31/2026

Assignment 1: Research Foundations for Software

SEN381

Authors:

Waldo Blom (578068)

Christian Janse van Rensburg (601840)



Contents

Question 1 - Software Engineering Beyond Programming	2
The interaction of Software Engineering concerns	2
Real World consequence: Knight Capital Group (2012)	2
Question 2 - Thinking Ahead: Decisions Across the Software Lifecycle	3
1. Stakeholder Roles and Responsibilities	3
2. User Requirements and Platform Selection (Mobile, Desktop or Web).....	3
3. Database Architecture Selection	4
4. Tech Stack selection.....	4
5. Test Planning and Quality Assurance.....	5
6. Deployment Architecture	5
Decision Horizon Map.....	7
Question 3 - Evidence-Based Technology Stack Decisions	9
Overview of Alternatives	9
Weighted Decision Matrix	9
Limitations of the matrix and contextual factors.	10
Critical question:	10
Question 4 - Responsible AI in Software Engineering.....	11
Three valuable uses of generative AI across Software Engineering activities	11
1. Using AI for Requirements Gathering.....	11
2. Code Completion and Suggestions.....	11
3. Generation of Automated Test Cases	11
Three significant risks or limitations associated with AI-assisted engineering.....	12
1. Lack of logic reasoning and understanding	12
2. Insecure Code and Security Vulnerabilities	12
3. Ethical considerations.....	12
Human verification / control process that should occur before AI-generated output becomes part of a software product	12
Accountability question	13
AI Research & Verification Record	14
References	15

Question 1 - Software Engineering Beyond Programming

The interaction of Software Engineering concerns

Successful software development is an engineering discipline expanding far beyond just writing code. According to the IEEE Software Engineering Body of Knowledge (IEEE Computer Society, 2024), software engineering involves solving real world problems cost effectively within complex constraints. Treating development just as “programming/coding” ignores these variables:

- Stakeholder needs and requirements define the systems business value. Executing code flawlessly is worthless if it builds the wrong Project/Scope.
- Scope, schedule, cost and resources form a constraint triangle. If an engineering team is forced to speed up the schedule or to reduce the resources without removing from the scope, developers are forced to “skimp” parts of the project.
- Cutting these corners degrades software quality, example, skipping automated testing or security checks.
- This degradation introduces technical debt, which can increase engineering risk such as operational failures, data breaches or unmaintainable code bases, which ultimately destroys stakeholder trust and inflate long term project cost.

Real World consequence: Knight Capital Group (2012)

On 1 August 2012, Knight Capital Group lost a lot of money due to a software error. The error occurred when a technician failed to copy new Retail Liquidity Program code to a server to meet a strict market deadline.

The decision: Under intense schedule pressure, engineers manually deployed the update to only 7 of 8 servers leaving a dormant 2005-era function called “Power Peg” active. The team bypassed automated deployment tools, skipped rigorous quality verification steps, and repurposed the obsolete “Power Peg” software flag instead of removing the dead code.

The trade off? The team traded their quality assurance and their engineering risk mitigation, such as testing and deployment verification, to meet a strict schedule, cost and resource constraint.

The consequence: When the system went live, the defective server didn’t recognize when orders were filled, this caused the algorithm to send more than 4 million erroneous orders into the market while trying to process only 212 real client orders. This resulted in a \$460 million loss in just 45 minutes, and the firm almost collapsed entirely (U.S. Securities and Exchange Commission, 2013).

Question 2 - Thinking Ahead: Decisions Across the Software Lifecycle

According to Kazman et al. a software engineering decision or concern is defined a choice within the software development lifecycle of a project that impacts the outcome of the project in some way.

These decisions occur across multiple development phases (planning, design, implementation, testing, deployment) and vary in importance and impact. Some decisions should be considered early because postponing them could constrain later choices or increase project risk or maybe even cause major rework, **some examples of choices that should be considered early are:**

1. Stakeholder Roles and Responsibilities

This decision has to do with defining the roles, responsibilities, required skills and communication channels for all the stakeholders in the project this includes the investors, end users, development team and project manager (Kazman et al.). It should be considered during project planning before any development begins (Kazman et al.). This decision impacts the overall end product (Kazman et al.). Without clear roles and communication channels it won't be possible to know who to hold responsible and who to contact when an issue arises making it impossible to resolve the problem (Kazman et al.).

In order to make this decision it is required to know the current implemented processes and the existing team skills, this can be achieved through developer interviews (Kazman et al.). Other factors include how we specifically define project success, and the estimated financial budget of the project. By ensuring that the team is capable and knowledgeable of this information it will allow the rest of SDCL process of the project to run smoothly and efficiently (Kazman et al.).

However, if this process is postponed it could cause major problems for example if there is not a team member that understands a specific technology required for the project this would require external contractors which is an unforeseen expense thereby increasing project cost. Additionally, unclear responsibility or communication channels could result in a person not knowing who to report which could lead to coordination problems and delayed decisions.

2. User Requirements and Platform Selection (Mobile, Desktop or Web)

User requirements need to be gathered when starting any software project, it needs to be determined what the business goals and end objectives of the system are because software architectures exist in order to satisfy these business goals (Kazman et al.). After determining exactly what the business requirements are, the development team can then decide what the primary platform(s) through which the end users will access and interact with the application (whether that is mobile devices, desktop computers, web browsers or a combination). It is considered operationally significant to consider how the end user will interact with the system because it shapes all the architectural design aspects of the system (Kazman et al.). Without clarity on how users will use and access the system it is not possible to effectively select appropriate technologies for development or decide on a development team (Kazman et al.).

In order to make this decision, the engineering and planning team must understand the end-user's context of how they are actually going to be using the software. This involves knowing whether the

users need a standalone desktop application, or a mobile app or maybe a browser-based application. This can be achieved by making use of what is known as quality attribute scenarios, which are documented descriptions of how the system must perform when the users interact with it on their screens (Kazman et al.). If done correctly it allows the development and planning team to set measurable expectations that align with the business needs of the project. This is discussed with the client, and feedback is incorporated through the established communication channels to ensure they are satisfied and that their business requirements have been met.

However, if this step is missed or delayed until development begins the project could face serious risks such as developers trying to force the use of a specific framework or language that doesn't fit the system's need (also referred to as architecture drift or architecture erosion) (Kazman et al.). This then leads to code that is difficult to understand and maintain (Kazman et al.).

3. Database Architecture Selection

Although this decision is influenced by the choice of the tech stack (as it is part of the stack), modern software engineering principles say that the database and core backend logic must be decoupled from the user interface (Kazman et al.). Thus, it is important to first start with the database architecture before the rest of the tech stack as it is a decision that involves choosing between a relational (SQL) and non-relational (NoSQL) database type (Khan et al.).

This decision is based on your data structure and application requirements (Khan et al.). It also involves the choice of selecting a specific database choice (for example, Microsoft SQL Server vs. PostgreSQL, or MongoDB vs. Firebase) (Khan et al.). Furthermore, it includes planning your backup and recovery strategy (Khan et al.). Thus, one decision (SQL vs. NOSQL) encompasses many other sub related decisions that should be made early in the design phase alongside the other choices of the tech stack.

In order to make this decision, you need information about your data structure (is the data structured, unstructured or semi structured) and the possible query patterns (how the application will search and get data from the database) (Khan et al.). It is also important to know how familiar the development team is with each database type and consider whether additional training or maybe even external resources would be needed (Kazman et al.). Selecting the appropriate database early enables the overall system performance, scalability and cost (Khan et al.). It also enables the abstraction layer to be isolated from the system correctly from the start.

Postponing this decision until later phases such as the development or testing phases introduces a risk: if the chosen database is not a correct one, a complete refactor would be needed (Kazman et al.). The cost of database migration increases with data volume, leading to unexpected delays and expenses (Kazman et al.). Additionally, mid-project database changes could require team retraining and extensive rework of data access layers in the codebase (Kazman et al.).

4. Tech Stack selection

After the database architecture is decided the next step is to decide the rest of the tech stack. In software engineering, a technology stack is defined as the combination of programming languages, frameworks, libraries, and developer tools used to construct and execute an application (Kumari, 2026). The three primary layers are : the frontend (what the end user sees and uses), the backend

(the actual logic and business rules related to data storage), and the database (where the data is stored) (Kazman et al.).

It should be considered during the design phase and should be an informed decision based on the development team capabilities as well as the platform the end users plan to access the platform on (Kumari, 2026). Other needed information includes project-specific requirements such as scalability, long term maintainability, project budget and licencing (some tech stacks are free, others you must pay for) (Kumari, 2026). Selecting a framework that is already established and supported early allows faster development, easier hiring, and reliable problem-solving (Kumari, 2026).

However, if this decision is postponed or not properly considered, you risk adopting a framework or technology that does not fit your needs and requirements. Also, adopting a brand-new framework without a lot of support has a risk of becoming obsolete, for example, FaunaDB's discontinuation left its early customers with projects that no longer worked and had to be reworked (Fireship, 2025). Mid project technology changes are extremely costly as they requiring code rewrites or potential team retraining / rehiring (Kazman et al.).

5. Test Planning and Quality Assurance

This decision has to do with defining how the testing will take place including unit testing, integration testing and acceptance testing. It essentially has to do with establishing a formal plan of how the software is going to be tested and how testing will be measured as a success which is achieved through formal test plans, procedures and reports, as well as the tools that will be used to actually test (NASA, 2022). It should be considered during the design phase after the tech stack has been decided as modern tech stacks sometimes includes built in testing frameworks such as Angular (Kumari, 2026). It is operationally significant because it directly affects detection of software bugs, reliability of the software and ultimately the end quality of the software (NASA, 2022).

In order to be able to make this decision, the engineering team needs to establish the following: quality standards for the application (by defining test coverage targets) and identify available testing tools. By establishing a clear testing strategy early, it will allow for quality assurance, faster deployment cycles, and earlier defect (bug) detection (NASA, 2022).

However, if testing strategy is postponed until late in development, quality issues will only be discovered later and might need major rework. This then leads to architecture erosion and drift which compromises the integrity of the end software product (Kazman et al.).

6. Deployment Architecture

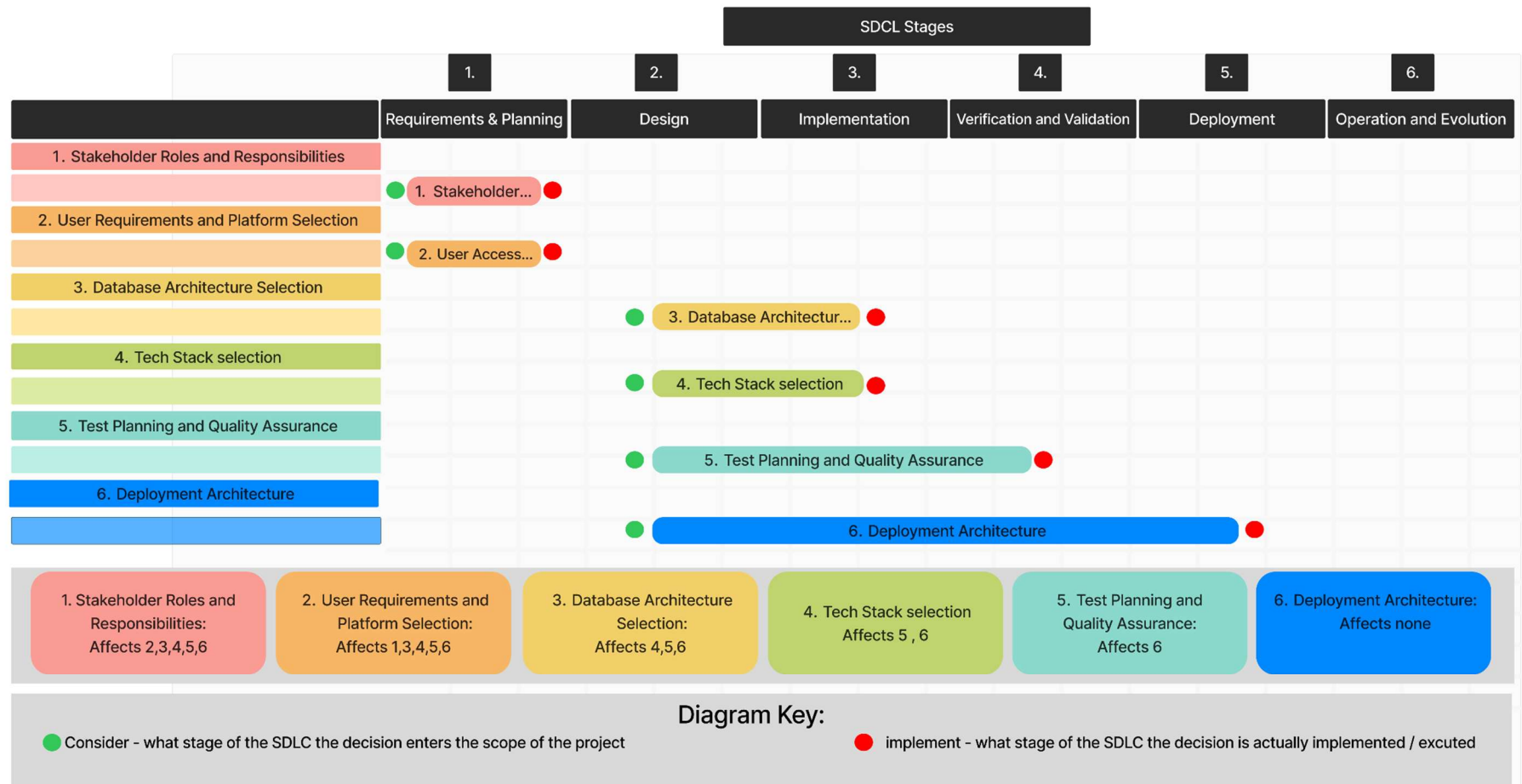
This decision involves selecting where and how the production project will be hosted, for example on cloud platforms (such as AWS or Azure), or on-premises infrastructure managed by the organization, or a hybrid combination of both (Younus et al.). It should be considered during design phase. It is operationally significant because it affects system reliability, operational costs, security, and the organization's long-term flexibility to scale or migrate (Younus et al).

In order to make this decision, the following information is needed: the allocated budget for deployment, how sensitive the data is that is being stored (on premises provides complete control of security of data and in cloud platforms you depend on their security), and the size and scope of the

application (Kazman et al.; Younus et al). For example, for a small web-based application it would not be justified to establish on-premises infrastructure, it would be cheaper and more efficient to use cloud-based hosting but if that infrastructure already exists and it can be used it might be more efficient to utilize the available resources. So, understanding what the current available infrastructure and resources are, is also important in determining the deployment option.

However, if this decision is delayed until the deployment stage of the SDLC it poses technical and financial risks to the project's long term viability (Kazman et al.). Without the correct deployment architecture planning, the application may not be able to support peak workloads, possibly causing system failures and data loss (Younus et al). Additionally, if an informed deployment decision is not made based on a complete understanding of budget constraints it could result in severe budget overruns and project delays (Kazman et al.).

Decision Horizon Map



Link to Figma diagram: <https://www.figma.com/design/JJiofE2WuMZorpw1bLIaOG/SEN381---Assignment-1--Question-2?m=auto&t=kMPxhOJMYJSGHava-1>

The chart above is influenced by a Gannt chart template

(<https://www.figma.com/community/file/1346166352943978767>)

The top part (the items in the black boxes), show the stages of the SDCL, the items on the left-hand side (the items coloured red, orange, yellow etc) indicate the SIX significant concerns. The green circles (dots) indicate the phase in which the SDCL the concern enters the scope of the project and the red circles (dots) indicate in which stage the concern is actually implemented / executed.

The diagram shows how each decision affects the next. It indicates that first the stakeholder roles and responsibilities need to be defined (who does what and how to communicate with each other), after the communication channels are established then user requirements can be discussed and the platform selection can be decided, then the requirements influence the database architecture, that in turn affects the rest of the tech stack, the tech stack influences the test planning and quality assurance, that then effects the deployment. From the diagram it is visible that the two most important decisions are at the top as they influence the rest of the project.

Question 3 - Evidence-Based Technology Stack Decisions

Overview of Alternatives

This comparison evaluates two approaches to a modern and scalable web-based information system. The purpose is to see how a technology decision should be made.

Stack A, JavaScript Stack: React frontend, Node.js backend and PostgreSQL.

Stack B, The .NET Stack: Angular frontend, C# backend and Microsoft SQL Server.

Weighted Decision Matrix

	Weight.	Stack A.	Stack B.	Justification of scores and evidence.
Team capability and learning curve.	15%	4/5	3/5	Stack A uses the popular language JavaScript which lowers the learning curve. Stack B in the other hand requires mastering C#, which could lead to having to hire external experts if the team lack knowledge.
Security ecosystem.	15%	3/5	5/5	Stack A relies on the npm ecosystem which is historically higher risk of supply chain vulnerabilities. Stack B relies on Microsoft's highly regulated NuGet ecosystem and identify frameworks.
Maintainability.	10%	4/5	5/5	Both Stacks are strong; however, Stack B is slightly better because ASP.NET enforces rigid architectural patterns out of the box which prevents spaghetti code.
Testing and tool support.	10%	4/5	5/5	Stack A requires configuring a fragmented ecosystem. Stack B has an integrated testing ecosystem, which is rooted within visual studio. (xUnit/NUnit)
Deployment/platform compatibility.	15%	5/5	4/5	Stack A is built for modern serverless networks like AWS, offering high flexibility. Stack B favours Platform as a Service/ Infrastructure as a Service like Azure App, which slightly lowers the score.
Development and operational cost.	15%	4/5	5/5	Stack A offers faster initial time to market. Stack B handles more requests per second on the same hardware, which lowers long term cloud costs. Which makes Stack B slightly more favourable in terms of cost.
Future change implications.	20%	3/5	5/5	Stack A relies on rapid evolving open-source libraries that many times introduces breaking changes.

				Stack B uses Microsoft LTS, which ensures decades of backwards compatibility and migration paths.
Total score.	100%	3.8	4.55	

Total score = $\sum$ (weight * score given to stack)

Limitations of the matrix and contextual factors.

While a decision matrix provides a good, structured evaluation, it mostly assumes all criteria are independent, which is rarely true in the real world. Context entirely dictates the outcome. Example, if a company's existing infrastructure is Linux based and the engineers only know JavaScript, Stack A becomes the correct choice regardless of Stack B's higher score.

Critical question:

Why is the highest-scoring or most popular technology stack not automatically the correct technology for every software project?

The highest-scoring stack is not always the correct option because software engineering is constrained by business realities and not benchmarks. A tech stack might score perfect on security and maintainability but if the development team does not have the necessary skills to use the stack, then the project will fail due to schedule and budget exhaustion. Historical failures where teams adopted popular but irrelevant tools, the best technology is the one that aligns with the specific requirements, team capabilities and deployment constraint of the project.

Question 4 - Responsible AI in Software Engineering

Specialized AI tools and Large Language Models (LLMs) can add value to software engineering, but at the same time they also come with risks. No matter to what extent these tools are used, they need to be used responsibly and there still needs to be some type of human intervention (Haque, 2025).

Three valuable uses of generative AI across Software Engineering activities

1. Using AI for Requirements Gathering

A project starts with requirements gathering, it starts with the developers and business stakeholders communicating with one another (Alenezi and Akour, 2025; Kazman et al.). This process is normally manual, it requires developers to gather information from many different sources such as through emails, meeting transcripts, and user feedback in order to identify what the software actually needs to do (the business requirements) (Alenezi and Akour, 2025; Kazman et al.). NLP (Natural Language processing) algorithms, such as GPT-4 and Gemini 1.5, can automatically process a large amount of unstructured data about what the stakeholders communicated in order to extract functional and non-functional requirements, identify logical inconsistencies, and detect any ambiguities before the development begins (Alenezi and Akour, 2025).

This approach could reduce the time spent on requirement analysis, but the main benefit is to identify any ambiguities and inconsistencies in order to resolve misunderstandings early to prevent problems occurring later in the other SDCL phases that are downstream from the requirements gathering.

2. Code Completion and Suggestions

Some IDE's (for example Visual Studio code) allow integration with tools such as GitHub Copilot in order to give context aware code suggestions that reduce the manual coding effort (Alenezi and Akour, 2025). Suggestions such as boiler plate code structures (loops or conditionals). It is important to note that this is not the same as complete code generation as it completes things such as syntax (Alenezi and Akour, 2025).

This allows developers to reduce time spent on routine, repetitive coding tasks thereby allowing them to spend more time on creating the actual complex logic (Alenezi and Akour, 2025). It could result in higher quality code and fewer bugs if used alongside applying critical thinking skills as the AI can make mistakes and is not always perfect and accurate (Alenezi and Akour, 2025).

3. Generation of Automated Test Cases

Testing is very important to software engineering as it ensures quality of the system and that the system works as intended (Alenezi and Akour, 2025; Kazman et al.). AI can be used to automate test case generation by analysing the code changes and predicting potential places where the software can fail (Alenezi and Akour, 2025). AI driven tools can automatically create unit tests and other testing to help identify possible missed edge cases (Alenezi and Akour, 2025).

By frequently updating the test cases, high-quality standards of the project can be ensured throughout the SDCL. It is however important to note that LLM's can often generate generic, redundant test cases so developers must still validate and refine the test cases before just accepting the proposed test cases (Alenezi and Akour, 2025).

Three significant risks or limitations associated with AI-assisted engineering

1. Lack of logic reasoning and understanding

AI tools such as LLMs operate only on statistical predication, based on patterns that were learned during training (Haque, 2025). The models lack logical reasoning or comprehension of the code that is generated (Haque, 2025). These AI models operate in what are referred to as “black boxes” meaning that their “decision making processes” are not easily interpretable by humans and most of the time not even actually visible to the user (Haque, 2025). AI can generate syntax correct code that compiles, but it is functionally incorrect and may include subtle incorrect logic (Haque, 2025). For example, a LLM could generate a syntactically correct code for a sorting algorithm but fail to account for implicit efficiency constraints like memory usage or time complexity (Haque, 2025).

2. Insecure Code and Security Vulnerabilities

As AI models are trained on very large public repositories that contain things such as outdated libraries or unpatched security vulnerabilities (Haque, 2025). When prompted, these AI tools can then inadvertently replicate these flaws (Haque, 2025). Examples include hard-coded credentials, weak encryption methods or not considering known patched vulnerabilities (Haque, 2025). If AI is not used responsibly alongside human intervention where needed this could cause major security risks especially in industries such as healthcare and banking (Haque, 2025).

3. Ethical considerations

According to (Haque, 2025) there are also ethical consideration due to the rise of use of AI tools, such as the impact on the work force, as it may reduce the demand for certain type of coding jobs. Particularly for entry level or junior developers (Haque, 2025). This could lead to job displacement and inequality within the industry, for example if no new junior developers are hired, how do they learn the particular system of a specific company (Haque, 2025). Additionally, there is also the risk on relying too dependently on LLMs and AI code generation tools, which could possibly lead to software engineers losing the ability to write complex code or solve problems independently (Haque, 2025). This could then affect the overall skills in the field and potentially the overall quality of software and innovation.

Human verification / control process that should occur before AI-generated output becomes part of a software product

From the previously mentioned above, it can be noted that AI tools and LLMS can add value but come with risks if output is not properly confirmed before it is accepted (Haque, 2025; Alenezi and Akour, 2025).

Human oversight of any output generated by AI is extremely important and necessary, this is often referred to as “Human-in-the-Loop” (Alenezi and Akour, 2025). This means that no output must immediately be accepted before there is some sort of human involvement and validation.

For example, when AI tool generates code, the developer should review and refine it in order to ensure that it fits the specified requirements and architecture of the system. If the developer uses AI for a specific bug fix, the developer should verify that the suggested solution actually resolves the problem rather than relying on the suggestion, because it appears possibly correct. Also, in terms of testing, AI-generated tests cases should be reviewed first to determine whether they actually test the required behaviour.

Accountability question

“If an engineer accepts AI-generated output that later introduces a defect, vulnerability or incorrect engineering decision, who remains accountable and why?”

The software engineer that accepted the AI generated output remains accountable because they made the choice to accept and integrate the proposed solution. AI should be used as tool and should not be treated as a person who has the final say in what is implemented, meaning that as mentioned previously that the engineers are responsible for reviewing, verifying and testing the output before approving it and integrating it into the actual software.

AI Research & Verification Record

Team member	AI tool / use	Purpose	Output used?	How independently verified	What was changed / rejected?
Christian	Gemini	Knight Capital case.	Yes	I read reports to make sure the facts and numbers about the stock crash were true.	I expanded on the information about the Knight Capital case.
Christian	Gemini	To help compare the two tech stacks.	Yes	I checked Microsoft official website to confirm their LTS lifespan.	I adjusted the reasons and stacks to fit our specific knowledge.
Waldo	Claude	To assist with finding research articles for Question 2 and Question 4. I prompted Claude to help suggest optimized search terms to get specific information e.g. of one used ``"AI assistance" NOT "replace" human accountability verification software engineering 2026`	Some search terms were used. Others were not. Some of them were modified.	I used the search term on google scholar. I then decided which articles were the creditable and matched my required research.	I adjusted some of the search terms, ignored the ones that would not result relevant information.

References

- Alenezi, Mamdouh, and Mohammed Akour. "AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions." *Applied Sciences*, vol. 15, no. 3, 2025, p. 1344, <https://www.mdpi.com/2076-3417/15/3/1344>, 10.3390/app15031344. Accessed 3 Sept. 2026.
- Fireship. *7 Programming Myths That Waste Your Time*. YouTube, 29 Apr. 2025.
- Haque, Md Asraful. "LLMs: A Game-Changer for Software Engineers?" *Benchmark Council Transactions on Benchmarks Standards and Evaluations*, vol. 5, no. 1, 1 Mar. 2025, pp. 100204–100204, 10.1016/j.tbench.2025.100204.
- IEEE Computer Society, 2024. Guide to the Software Engineering Body of Knowledge (SWEBOK Version 4.0). Los Alamitos: IEEE Computer Society.
- Kazman, Rick, et al. "A Holistic View of Architecture Definition, Evolution, and Analysis." *Carnegie Mellon University*, 2023, https://www.sei.cmu.edu/documents/5720/2023_005_001_983542.pdf, 10.1184/R1/20722402. Accessed 1 Sept. 2026.
- Khan, Wisal, et al. "SQL and NoSQL Database Software Architecture Performance Analysis and Assessments—a Systematic Literature Review." *Big Data and Cognitive Computing*, vol. 7, no. 2, 1 June 2023, p. 97. *MDPI*, <https://www.mdpi.com/2504-2289/7/2/97>. Accessed 3 Sept. 2026.
- Kumari, Tanya. "Classic Informatics: Top Web Development Company in India." *Classicinfo*, 3 July 2026, <https://www.classicinformatics.com/blog/how-to-choose-the-right-tech-stack>. Accessed 2 Sept. 2026.
- NASA. "NPR 7150.2D - Chapter4." *Nodis3.Gsfc.Nasa.Gov*, 8 Mar. 2022, https://nodis3.gsfc.nasa.gov/displayDir.cfm?Internal_ID=N_PR_7150_002D_&page_name=Chapter4. Accessed 2 Sept. 2026.
- U.S. Securities and Exchange Commission, 2013. Securities Exchange Act of 1934 Release No. 70694. [online] SEC.gov. Available at: <https://www.sec.gov/litigation/admin/2013/34-70694.pdf> [Accessed 2 September 2026].
- Younus, Salman, et al. "Systematic Analysis of on Premise and Cloud Services." *International Journal of Cloud Computing*, vol. 13, no. 3, 1 Jan. 2024, 10.1504/ijcc.2024.10063641.